

Vertex Logging Framework Specification

Version: 1.0

Status: Draft

1. Objetivo

O Logging Framework do Vertex é o subsistema responsável por capturar, estruturar, armazenar e expor logs de todo o sistema.

Seu objetivo é fornecer rastreabilidade completa de eventos internos, execução de plugins, decisões do Core e comportamento de subsistemas, com baixo impacto de performance.

Ele funciona como a camada central de observabilidade do Vertex.

2. Responsabilidades

O Logging Framework é responsável por:

- registrar logs de Core e plugins;
- classificar níveis de severidade;
- estruturar logs de forma consistente;
- armazenar logs com eficiência;
- integrar logs entre subsistemas;
- fornecer consulta e filtragem;
- suportar logs em runtime;
- garantir baixo impacto de performance;
- exportar logs para diagnóstico;
- suportar auditoria de segurança.

3. Arquitetura

System / Plugin

↓

Logging API

↓

Logging Framework

- Log Collector
- Log Router
- Log Buffer
- Log Formatter
- Log Storage Engine
- Log Level Manager
- Log Filter Engine
- Log Exporter
- Metrics Collector

↓

Storage Engine / External Sink

4. Componentes

Log Collector

Recebe logs de todos os subsistemas.

Log Router

Direciona logs para armazenamento ou exportação.

Log Buffer

Armazena logs temporariamente para otimização de performance.

Log Formatter

Estrutura logs em formato consistente (JSON-like estruturado).

Log Storage Engine

Persistência de logs no Storage Engine.

Log Level Manager

Gerencia níveis de log:

- DEBUG
- INFO
- WARN
- ERROR
- CRITICAL

Log Filter Engine

Filtra logs por:

- plugin;
- subsistema;
- severidade;
- tempo;
- contexto.

Log Exporter

Exporta logs para diagnóstico externo ou ferramentas internas.

Metrics Collector

Mede volume e performance de logging.

5. Fluxos

Registro de log

Plugin / Core

↓

Logging API

↓

Log Collector



Log Formatter



Log Buffer



Storage / Export

Consulta de logs

Query Request



Log Filter Engine



Log Storage Engine



Result Set

Exportação

Log Buffer / Storage



Log Exporter



External Tool / System

Overflow de buffer

Log Buffer Full

↓

Log Router

↓

Storage Engine (Flush)

↓

Buffer Cleared

6. APIs

Registro

- logDebug()
- logInfo()
- logWarn()
- logError()
- logCritical()

Consulta

- queryLogs()
- getLogsByPlugin()
- getLogsByTimeRange()

Controle

- setLogLevel()
- enableLogging()
- disableLogging()

Exportação

- exportLogs()
- streamLogs()

7. Estados

Estado do sistema de logs

Active

↓

Buffered

↓

Flushing

↓

Persisted

↓

Exporting

↓

Disabled

Estado de logs individuais

- Recorded
- Buffered
- Persisted
- Exported
- Dropped

8. Políticas

Baixo impacto de performance

Logging nunca deve bloquear execução principal.

Estrutura obrigatória

Logs devem ser estruturados e não apenas texto bruto.

Bufferização

Logs podem ser armazenados temporariamente em buffer.

Prioridade de segurança

Logs críticos não podem ser descartados.

Controle de volume

Sistema pode reduzir logs em modo de baixa performance.

Isolamento por plugin

Cada plugin possui contexto próprio de logs.

9. Integrações

O Logging Framework integra-se com:

- Execution Runtime;
- Scheduler;
- Plugin Manager;
- Plugin Context;
- Security Framework;
- Resource Manager;
- Memory Management;
- State Engine;
- Event System;
- Storage Engine;
- Configuration System;

- UI Runtime (debug views);
- Boot Manager;
- Power Management.

10. Métricas

O sistema coleta:

- volume de logs por segundo;
- logs por plugin;
- taxa de erro;
- uso de buffer;
- latência de escrita;
- logs descartados;
- consumo de memória do logging;
- tempo de flush;
- impacto em performance.

11. Exemplos

Log de execução de plugin

1. Plugin inicia execução.
2. Logging API registra INFO.
3. Log Buffer armazena temporariamente.
4. Log é persistido no Storage Engine.

Erro crítico no sistema

1. Exception ocorre no Execution Runtime.
2. Log Critical é gerado.
3. Log é imediatamente persistido.
4. Event System pode notificar subsistemas.

Sobrecarga de logs

1. Muitos logs DEBUG são gerados.
2. Log Level Manager reduz nível automaticamente.
3. Buffer evita impacto no sistema.
4. Logs menos importantes são descartados.

Consulta de diagnóstico

1. Desenvolvedor chama queryLogs().
2. Log Filter Engine processa filtros.
3. Logs relevantes são retornados.
4. UI Debug Viewer exibe resultado.

12. Regras Arquiteturais

- Logging nunca deve bloquear execução principal.
- Logs devem ser estruturados.
- Logs críticos nunca podem ser perdidos.
- Cada plugin deve ter isolamento de logs.
- O sistema deve se adaptar à carga automaticamente.
- Logs devem ser rastreáveis e filtráveis.

13. Limitações

O Logging Framework não é responsável por:

- execução de código;
- armazenamento de dados de aplicação;
- gerenciamento de UI;
- agendamento de tarefas;
- controle de recursos.

14. Futuras Extensões

O Logging Framework poderá evoluir para:

- logs distribuídos entre dispositivos Vertex;
- análise de logs por IA;
- detecção automática de anomalias;
- compressão avançada de logs;
- tracing completo de execução end-to-end;
- visualização em tempo real no UI Runtime.

15. Resumo

O Logging Framework é a camada de observabilidade do Vertex. Ele captura, organiza e expõe logs de todo o sistema de forma eficiente e estruturada, garantindo rastreabilidade sem comprometer desempenho. Em conjunto com Security Framework e Event System, forma a base de diagnóstico e auditoria da plataforma.